

GrowEasy AI CSV Importer

Complete Project Explanation & Submission Guide

Project: AI-Powered CSV Lead Importer

Company: GrowEasy (<https://growseasy.ai>)

Assignment Role: Software Developer Intern / Full-Time

Tech Stack: Next.js - Node.js - Express - Google Gemini - TypeScript - Tailwind CSS

Local Path: `~/Desktop/growseasy-csv-importer`

PART 1 - How to Share / Submit This App

Assignment asks you to email **varun@growseasy.ai** with:

1. Hosted application URL
2. GitHub repository URL
3. Position applied for (Intern OR Full-Time)

Deadline mentioned in assignment: **11 July 2026, 6 PM**

Step A - Push code to GitHub

```
cd ~/Desktop/growseasy-csv-importer
```

```
# IMPORTANT: never commit API keys
```

```
# Confirm .env is in .gitignore (already done)
```

```
git init
```

```
git add .
```

```
git status # make sure backend/.env is NOT listed
```

```
git commit -m "AI-powered CSV lead importer for GrowEasy assignment"
```

```
# Create a new public repo on github.com, then:
```

```
git branch -M main
```

```
git remote add origin https://github.com/YOUR_USERNAME/growseasy-csv-importer.git
```

```
git push -u origin main
```

Public repo URL example:

```
`https://github.com/YOUR_USERNAME/growseasy-csv-importer`
```

Step B - Host the application (required)

You need the app live on the internet (not only localhost).

Recommended setup

| Part | Platform | Why |

|-----|-----|-----|

| Frontend (Next.js) | **Vercel** | Free, easy for Next.js |

| Backend (Express) | **Railway** or **Render** | Free tier for Node APIs |

Frontend on Vercel

1. Go to <https://vercel.com> -> Sign in with GitHub
2. Import `growseasy-csv-importer` repo
3. Set **Root Directory** = `frontend`

4. Environment variable:

- `NEXT\_PUBLIC\_API\_URL` = your backend URL

Example: `https://groweasy-api.onrender.com`

5. Deploy -> you get a URL like `https://groweasy-csv-importer.vercel.app`

Backend on Render (example)

1. Go to <https://render.com> -> New Web Service

2. Connect same GitHub repo

3. Settings:

- Root Directory: `backend`

- Build Command: `npm install && npm run build`

- Start Command: `npm start`

4. Environment variables:

- `GEMINI\_API\_KEY` = your Gemini key

- `GEMINI\_MODEL` = `gemini-2.0-flash`

- `CORS\_ORIGIN` = your Vercel URL

- `PORT` = `4000` (or whatever Render provides)

5. Deploy -> copy backend URL into Vercel `NEXT\_PUBLIC\_API\_URL`, redeploy frontend

Alternative: Railway

Same idea - deploy `backend` folder as a service, set env vars, copy public URL.

Step C - Email template (copy-paste)

To: varun@groweasy.ai

Subject: GrowEasy Assignment Submission - Software Developer Intern

Hi Varun,

Please find my submission for the GrowEasy Software Developer assignment.

Position applied for: Software Developer Intern

(or: Software Developer Full-Time)

Hosted application URL:

<https://YOUR-APP.vercel.app>

GitHub repository URL:

https://github.com/YOUR_USERNAME/groweasy-csv-importer

Brief summary:

I built an AI-powered CSV importer using Next.js (frontend) and Node.js + Express (backend). Users upload any CSV, preview rows, confirm import, and Google Gemini maps arbitrary columns into GrowEasy CRM fields in batches, with progress tracking and skipped-record handling.

Thank you,

YOUR NAME

YOUR PHONE

YOUR EMAIL

Step D - Before sending checklist

- [] App opens on hosted URL (not only localhost)

- [] CSV upload -> preview -> confirm -> AI results works live

- [] GitHub repo is **public**
- [] README has setup instructions
- [] `.env` / API keys are **NOT** committed
- [] Position (Intern / Full-Time) clearly mentioned
- [] Email sent before deadline

PART 2 - What This App Does (Simple Explanation)

Problem

Different companies export leads in different CSV formats:

- Facebook: ``Full Name`, `Email Address`, `Phone Number``
- Google Ads: different headers
- Excel / agency sheets: messy columns

GrowEasy CRM needs fixed fields like ``name`, `email`, `country_code`, `crm_status``, etc.

Challenge is NOT reading CSV.

Challenge is intelligently mapping unknown columns using AI.

Solution

1. User uploads any valid CSV
2. App shows a preview table (no AI yet)
3. User clicks Confirm Import
4. Backend sends rows to Gemini AI in batches
5. AI returns structured GrowEasy CRM JSON
6. UI shows successfully imported + skipped records

PART 3 - Tools & Technologies Used

Frontend

Tool	Purpose
Next.js 15	React framework (App Router) - assignment requirement
React 19	UI components
TypeScript	Type safety
Tailwind CSS	Modern responsive styling
PapaParse	Parse CSV in the browser for preview
Google Fonts	(Outfit + DM Sans) Clean typography

Backend

Tool	Purpose
Node.js	Runtime - assignment requirement
Express	REST API server - assignment requirement
TypeScript + tsx	Typed backend + hot reload
Multer	Multipart file upload support
PapaParse	Server-side CSV parsing
dotenv	Environment variables (API keys)

| ****cors**** | Allow frontend <-> backend calls |

AI

| Tool | Purpose |

|-----|-----|

| ****Google Gemini**** (`gemini-2.0-flash`) | LLM for field mapping |

| ****@google/generative-ai**** | Official Gemini SDK |

| Custom prompt engineering | CRM rules, enums, skip logic |

Extra / Bonus

| Tool | Purpose |

|-----|-----|

| Batch processing | Send rows in chunks (e.g. 15) |

| Retry mechanism | Re-try failed AI batches |

| Async job + progress polling | Progress bar in UI |

| Heuristic demo mapper | Works offline if key missing (local only) |

| Unit tests | CSV parse + phone/status normalize |

| Docker / docker-compose | Containerized run |

| Sample CSVs | Clean + messy test files |

PART 4 - How We Built It (Step by Step)

1. Project structure

```
groweasy-csv-importer/
+-- frontend/                # Next.js UI (port 3000)
|   +-- src/app/page.tsx     # Main screen + import flow
|   +-- src/components/      # Sidebar, Modal, Upload, Preview, Results
|   +-- src/lib/csv.ts       # Parse CSV + call APIs
+-- backend/                 # Express API (port 4000)
|   +-- src/routes/          # API endpoints
|   +-- src/services/        # AI + import jobs
|   +-- src/utils/           # CSV parse + normalize
+-- sample-data/             # Test CSV files
+-- docker-compose.yml
+-- README.md
```

2. Frontend flow implementation

Step 1 - Upload

- Component: `UploadDropzone.tsx`
- Drag & drop + click to browse
- Validates `.csv` and max 5MB
- Uses PapaParse to parse locally

Step 2 - Preview

- Component: `PreviewTable.tsx`
- Shows filename, size, row count
- Responsive table with sticky headers
- Horizontal + vertical scroll
- ****No AI call here**** (as required)

Step 3 - Confirm

- Button: **Confirm Import**
- Calls `POST /api/import/async` with parsed rows
- Shows processing modal + progress

Step 4 - Results

- Component: `ResultsView.tsx`
- Imported CRM leads table
- Skipped records + reasons
- Totals: imported / skipped / processed

3. Backend flow implementation

1. Accept rows (JSON) or CSV file
2. Split into batches (`AI_BATCH_SIZE`, default 15)
3. For each batch -> Gemini prompt with CRM rules
4. Normalize response (phone split, status enums, dates)
5. Skip rows with neither email nor mobile
6. Return structured JSON + progress updates

4. AI prompt design (important for evaluation)

The prompt tells Gemini to:

- Map any header names semantically (`Full Name` -> `name`, etc.)
- Use only allowed `crm_status` values
- Use only allowed `data_source` values (or blank)
- Split phone into `country_code` + `mobile_without_country_code`
- Put extra emails/phones into `crm_note`
- Skip if no email and no mobile
- Return strict JSON only

This is the core of **AI Prompt Engineering** scoring.

PART 5 - CRM Fields Extracted

Field	Meaning
created_at	Lead creation date/time
name	Lead name
email	Primary email
country_code	e.g. +91
mobile_without_country_code	Mobile digits only
company	Company name
city / state / country	Location
lead_owner	Owner
crm_status	Allowed status enum
crm_note	Notes / extras
data_source	Allowed source enum or blank
possession_time	Property possession
description	Extra description

Allowed `crm_status`

- `GOOD_LEAD_FOLLOW_UP`
- `DID_NOT_CONNECT`

- `BAD\_LEAD`
- `SALE\_DONE`

Allowed data_source

- `leads\_on\_demand`
- `meridian\_tower`
- `eden\_park`
- `varah\_swamy`
- `sarjapur\_plots`
- (or blank if unsure)

PART 6 - API Reference

Method	Endpoint	Description
-----	-----	-----
GET	`/health`	Health + AI/demo mode
POST	`/api/preview`	Parse CSV only (no AI)
POST	`/api/import`	Sync full AI import
POST	`/api/import/async`	Start async job, returns jobId
GET	`/api/import/:id/progress`	Progress / batch status
GET	`/api/import/:id/results`	Final imported + skipped

PART 7 - How to Run Locally

Prerequisites

- Node.js 18+
- Gemini API key from <https://aistudio.google.com/apikey>

Setup

```
cd ~/Desktop/groweasy-csv-importer
npm run install:all

cp backend/.env.example backend/.env
# Edit backend/.env -> set GEMINI_API_KEY=your_key

cp frontend/.env.example frontend/.env.local
# NEXT_PUBLIC_API_URL=http://localhost:4000
```

Run (2 terminals)

```
# Terminal 1
npm run dev:backend

# Terminal 2
npm run dev:frontend
```

Open: **http://localhost:3000**

Test file: `sample-data/messy\_facebook\_export.csv`

PART 8 - What to Say in Interview / Demo

30-second pitch

> I built an AI-powered CSV importer for GrowEasy. The frontend is Next.js with drag-and-drop upload and a preview table. After the user confirms, an Express backend sends the rows to Google Gemini in batches. The model maps any CSV columns into GrowEasy CRM fields, we normalize phones and statuses, skip invalid rows, and show imported vs skipped results with a progress indicator.

If they ask ?Why Gemini??

> Assignment allows OpenAI, Gemini, or Claude. I used Gemini because it has a free developer key, fast flash model, and good JSON responses for structured extraction.

If they ask ?What was hardest??

> Handling messy headers and inconsistent phone/email formats. Prompt engineering plus a normalization layer (split country code, status aliases, skip rules) made results reliable.

If they ask ?How do you scale??

> Batch processing, retries, async jobs with progress polling. For production we could add queues (BullMQ), rate-limit handling, and virtualized tables for very large CSVs.

PART 9 - Evaluation Mapping (Why This Scores Well)

| Criteria | How we covered it |

|-----|-----|

| AI Prompt Engineering | Detailed CRM rules prompt + enums + skip logic |

| Backend Quality | Express routes, batches, retries, error handling |

| Frontend Quality | Modern UI, preview, loading, responsive tables |

| Code Quality | TypeScript, clean folders, reusable components |

| Bonus | Drag & drop, progress, retries, Docker, tests, README |

PART 10 - Security Notes

- Keep `GEMINI\_API\_KEY` only in `.env` / hosting env vars
- Never commit `.env` to GitHub
- If a key was shared in chat, regenerate it in Google AI Studio after submission
- CORS should allow only your frontend domain in production

PART 11 - Final Submission Summary

****What is done in code****

- Full frontend import UX
- Full backend AI pipeline
- Sample data, README, Docker, tests

****What you still must do to submit****

1. Push to public GitHub
2. Deploy frontend + backend
3. Email hosted URL + GitHub URL + position to varun@groweasy.ai

****Document prepared for GrowEasy assignment submission & viva/explanation.****

****Project folder:** `Desktop/groweasy-csv-importer`**